

Document number: P3541R1
Date: 2025-01-06
Audience: SG21, SG23, EWG
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Violation handlers vs noexcept

In this paper we point out that a number of correctness-checking features currently considered for addition to C++ that allow responding to run-time-detected violations via exceptions, should define the interaction with the `noexcept`-specifier, with the `noexcept`-operator, and also specify what can be expected of stack unwinding in the case of a detected violation.

0. Revision history

0.1. R0 → R1

- Apart from `noexcept` we now also consider stack unwinding guarantees.
- Restructured the discussion to make it clear that the `noexcept`-operator and the `noexcept`-specifier are two separate issues.

1. The context

There is a number of proposals currently on the flight aiming at detecting at run-time the incorrect usages of the language or libraries, and responding to them in a number of ways, including by throwing an exception. These proposals include:

1. [\[P2900R11\]](#) — Contracts for C++,
2. [\[P3471R2\]](#) — Standard library hardening,
3. [\[P3100R1\]](#) — Undefined and erroneous behaviour is a contract violation,
4. [\[P3081R0\]](#) — Core safety profiles.

[\[P2900R11\]](#) introduces the notion of a *violation handler*: a user-customisable function that is invoked in response to a violation of a user-planted *contract assertion* detected at run-time. One of the intended use cases for this handler is for the programmer to be able to throw an exception that would on the one hand prevent the execution of the following expression, and on the other resume the program execution from another location.

[\[P3471R2\]](#) wants to employ the same violation handler mechanism for reporting the violations of preconditions of certain Standard Library functions (not necessarily expressed with contract assertions). [\[P3100R1\]](#) proposes (among other things) to put an *implicit* contract assertion (which may end up calling a violation handler) in front of every expression with a potential to trigger an *easily detectable undefined behavior*, such as null pointer dereference or integer division by zero. Finally, [\[P3081R0\]](#) proposes that under certain predefined profiles, a subset of expressions with easily detectable UB situations from [\[P3100R1\]](#) — namely, null pointer dereference, element access operator call and integer overflow — be preceded by an implicit contract assertion.

In case [\[P2900R11\]](#) isn't accepted for C++26, [\[P3471R2\]](#)'s plan B is to just terminate (in an unspecified way) and there will be no question of throwing. [\[P3081R0\]](#)'s plan B is to devise its own mechanism of a contract violation handler, with the same set of questions to be addressed.

2. The use case

There are two use cases for throwing violation handlers.

1. To unit-tests the presence of run-time correctness checks in the body of a function.
2. To restore a program that was about to execute an operation out of contract, to a state where it can correctly continue the execution.

The first case has been described in detail in [\[N3248\]](#), is well understood, and only applies to unit tests, so programs that can afford some glitches like memory leaks. In this paper we will be focusing on the second case.

We will call this use case *recover-from-a-bug*.

In order to explain it we will introduce another term: *imminent undefined behavior*. Under some proposed features an implementation may insert a run-time check before a given expression *E* that determines, by evaluating a boolean predicate, if the evaluation of *E* would result in undefined behavior. If this run-time check detects that the undefined behavior would take place, we call this run-time situation an imminent undefined behavior, but it is not undefined behavior just yet. For example, under proposal [\[P3100R1\]](#) an implementation inserts before an expression **p* a run-time check if *p* is null. If this run-time check at some point discovers that *p* is indeed null, this is imminent undefined behavior. But expression **p* has not yet executed, and may never will because the implementation can also insert an instruction (such as program termination) that will cause the evaluation of the expression never to be reached.

Now, the use case: Upon an imminent undefined behavior or a contract violation we want the control to be transferred to the higher layers of the program which can decide how to clean up the critical resources and reset the state of the program. We cannot afford to terminate the program and rely on an external process to re-run it, because our program is the only process that is running on the hardware.

Because the detection of the imminent undefined behavior or contract violation is an indication of a software bug that has happened *earlier* in the program execution, there is a chance that the bug originates in the higher layers of the program, and such a restart will not work. So the engineers take a *chance*.

3. The problems

3.1. Stack unwinding for unannounced throw

In this paper we use term *unannounced throw* to refer to a throw from a violation handler.

Given the following code

```
void set_positive(int* pValue)
{
    std::lock_guard _ {_mutex};
    _result = (*pValue > 0); // i
}
```

if the pointer dereference ends in a throw, as per [\[P3081R0\]](#), is it guaranteed that `_mutex` is unlocked? Today, in C++23, the expression statement denoted with *i* cannot throw, so the implementation can just emit a call to `unlock` as a regular instruction, without caring if the instruction is also invoked upon stack unwinding.

In the [\[P3081R0\]](#) world, either the destructor call is guaranteed upon unannounced throw, and there is a run-time penalty to be paid even in the correct code (either as a longer execution time or as a bigger executable), or we have a different kind of stack unwinding that doesn't guarantee that destructors are called. Or maybe, rather than having the implementation choose between just two — calling the violation handler and not calling the handler

— we should have it choose between three: not calling it, calling it with normal stack unwinding upon throw, and calling it with the degenerate stack unwinding upon throw?

In a similar vein, a code that guarantees *strong* (transactional) exception safety ([\[ABRAHAMS\]](#)), which relies on some operations never throwing, may no longer work for these new unannounced throws.

An argument is often brought up that these new unannounced throws would only show up in places where we had undefined behavior before, so the unannounced throw is no worse than the previous UB. However, such attitude is not enough to satisfy the use case *recover-from-a-bug*. After all, we are talking about a change from something that used to be undefined behavior into something that has a well defined behavior. Proposals like [\[P3081R0\]](#) or [\[P3100R1\]](#) should make it clear if they want to support the use case, and if they do, specify clearly how the program behaves in the face of such unannounced throws.

3.2. Hitting the `noexcept` barrier

The unannounced throw which can now happen nearly anywhere can cause an exception to escape from a `noexcept` function. Note that destructors are implicitly annotated as `noexcept`. Consider:

```
Tool::~~Tool() // noexcept by default
{
    for (int i = 0; i <= size(); ++i) {
        _vector[i].retire(i);
    }
}
```

If the out-of-bounds access is detected and turned into a throw, this will cause `std::terminate()` to be called, and will compromise the *recover-from-a-bug* use case.

Solutions proposed in [\[P2784R0\]](#) and [\[P3205R0\]](#) offer a way to perform sort-of-a-stack-unwinding that can propagate across `noexcept` barriers while skipping the destructor calls.

3.3. The `noexcept` operator

In C++23 the value of expression

```
noexcept(p->clear())
```

is `true`. What should be its value under [\[P3081R0\]](#) or [\[P3100R1\]](#), given that the null pointer dereference can call a violation handler that throws?

The possible answers to this question, partially overlapping with these from [\[P2969R0\]](#), are:

1. Any construct that might end up in either undefined behavior or contract violation is potentially-throwing and this is reflected by the `noexcept`-operator. This goes against the direction in [\[P0709R4\]](#) or in [\[P3166R0\]](#).
2. All these constructs are non-throwing, reflected by the `noexcept`-operator, and we accept that throwing from a violation handler is disallowed.
3. The said constructs are either observably non-throwing or observably potentially-throwing based on some configuration, maybe a compiler switch. The result is that a correct program can take different paths based on configuration.
4. The said contracts are `noexcept(true)` but can throw nonetheless in case the violation handler throws. This would set the meaning of `noexcept` to "expression does not throw as long as it does not violate any contract".

5. Keep avoiding the answer for [\[P2900R11\]](#). For the imminent undefined behavior cases allow only semantics *ignore* and *quick_enforce*, which have no means of causing an exception to be thrown. This can be later expanded to semantics like *observe* and *enforce* but which turn a throw into a program termination.

4. The interaction with contracts

Contracts ([\[P2900R11\]](#)) are special in this regard, as the run-time checks are never added implicitly but require an explicit usage of either `pre` or `post` on function declarations or `contract_assert` as a statement.

[\[P2900R11\]](#) clearly defines the interaction with `noexcept`. Preconditions and postconditions, even if physically checked in the caller, interact with `noexcept` as if they were evaluated inside the function body.

`contract_assert` is a statement rather than an expression, so that its non-throwing property cannot be tested in any way.

However, while this works for now, we know of two features that propose to be able to test the non-throwing property of arbitrary statements:

1. [\[P3166R0\]](#) — *Static Exception Specifications*, where you can annotate your function with `throw(auto)` which will deduce the exception specification from all statements in the function body,
2. [\[P2806R2\]](#) — *do expressions*, which turn a sequence of statements into an expression which you can inspect with the `noexcept`-operator.

If either of these features is added, the question will need to be answered: is `contract_assert` potentially-throwing?

The evaluation of `contract_assert` could throw for two reasons: either because the violation handler throws, or because the tested predicate throws. If the direction should be that assertion statements are non-throwing, we could end up with an interesting result that

```
noexcept(contract_assert((throw 1, false)))
```

returns `true`.

5. Conclusion

Given that the same idea — potentially throwing in a controlled way from every place that is either an imminent undefined behavior or a contract violation — surfaces in a number of proposals, the C++ community would benefit from a clear design direction in this respect.

Any proposal in this space should clearly state if use case *recover-from-a-bug* is supported, and how: what is guaranteed and what is not.

6. Acknowledgments

A number of observations in this paper comes from SG21 reflector participants, in particular Ville Voutilainen and Anthony Williams. Timur Doumler and Herb Sutter suggested improvements and corrections to the paper. The list of suggested solutions comes from [\[P2969R0\]](#).

7. References

- [ABRAHAMS] — David Abrahams, "Exception-Safety in Generic Components" (https://www.boost.org/community/exception_safety.html).
- [N2855] — Douglas Gregor, David Abrahams, "Rvalue References and Exception Safety" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2009/n2855.html>).
- [N2983] — David Abrahams, Rani Sharoni, Doug Gregor, "Allowing Move Constructors to Throw" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2009/n2983.html>).
- [N3248] — Alisdair Meredith, John Lakos, "noexcept Prevents Library Validation" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2011/n3248.pdf>).
- [P0709R4] — Herb Sutter, "Zero-overhead deterministic exceptions: Throwing values" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0709r4.pdf>).
- [P2698R0] — Bjarne Stroustrup, "Unconditional termination is a serious problem" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2698r0.pdf>).
- [P2780R0] — Ville Voutilainen, "Caller-side precondition checking, and Eval_and_throw" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2780r0.html>).
- [P2784R0] — Andrzej Krzemiński, "Not halting the program after detected contract violation" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2784r0.html>).
- [P2806R2] — Bruno Cardoso Lopes, Zach Laine, Michael Park, Barry Revzin, "do expressions" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2806r2.html>).
- [P2858R0] — Andrzej Krzemiński, "Noexcept vs contract violations", (<https://isocpp.org/files/papers/P2858R0.html>).
- [P2900R11] — Joshua Berne, Timur Doumler, Andrzej Krzemiński et al., "Contracts for C++", (<https://isocpp.org/files/papers/P2900R11.pdf>).
- [P2969R0] — Timur Doumler, Ville Voutilainen, Tom Honermann, "Contract annotations are potentially-throwing" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2969r0.pdf>).
- [P3081R0] — Herb Sutter, "Core safety Profiles: Specification, adoptability, and impact" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3081r0.pdf>).
- [P3100R1] — Timur Doumler, Gašper Ažman, Joshua Berne, "Undefined and erroneous behaviour is a contract violation" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3100r1.pdf>).
- [P3113R0] — Timur Doumler, "Contract assertions, the noexcept operator, and deduced exception specifications" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3113r0.pdf>).
- [P3114R0] — Andrzej Krzemiński, "noexcept(contract_assert(_))" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3114r0.pdf>).
- [P3166R0] — Lewis Baker, "Static Exception Specifications" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3166r0.html>).
- [P3205R0] — Gašper Ažman, Jeff Snyder, Andrei Zissu, Ben Craig, "Throwing from a noexcept function should be a contract violation." (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3205r0.pdf>).
- [P3471R2] — Konstantin Varlamov, Louis Dionne, "Standard library hardening" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3471r2.html>).